

CS3754 Cloud Software Development

Semester Project Report

Fall 2024

Feather Finder

Logan Kemprowski
Chaebin Kim
Hamza Iqbal
Samarth Jain
Quang Dao Le
Solaiman Ibrahim

Department of Computer Science
Virginia Tech
Blacksburg, VA 24061

App's URL: <http://3.128.28.199:8080/FeatherFinder/index.xhtml>

Date: 10/25/24
Team Number: 15
Instructor: [Prof. Osman Balci](#)

USER ACCOUNTS

Using your software application running on your Amazon EC2 VPS, each team member is required to create an account and generate *meaningful content* in the database for evaluation and testing. **You will be penalized for not having sufficient content.**

In the case of sign-in with two-factor authentication (2FA), set each user account below with 2FA turned off for grading.

List the usernames and passwords below to use for grading the app.

<i>Username</i>	<i>Password</i>
hamzai21	Birdsapp@2024
logank55	Birds@123
ckk111	Finder123!
kuplar	Chocolate1*
thanos79	H@ppro79
samarthjain2023	CSD@mysql-1872

EXECUTIVE SUMMARY

We worked on the Feather Finder project, developed by Team 15 for the CS3754 Cloud Software Development course at Virginia Tech. Feather Finder is a Jakarta EE cloud-based web application designed to enhance the birdwatching experience for explorers, hikers, and bird enthusiasts. By integrating APIs such as the eBird, Nuthatch, Gemini, and Google Maps API, the application lets a user identify, log, and track bird sightings and species in an all in one package by compiling all these different resources into one application. Key features of the application include a personal bird sightings database, allowing users to keep track of the birds that they have identified alongside information such as specific observation notes and time of sighting, interactive maps for seeing where a bird was spotted and when, an AI-powered chatbot for bird-related queries, and a leaderboard feature to encourage user engagement through friendly competition.

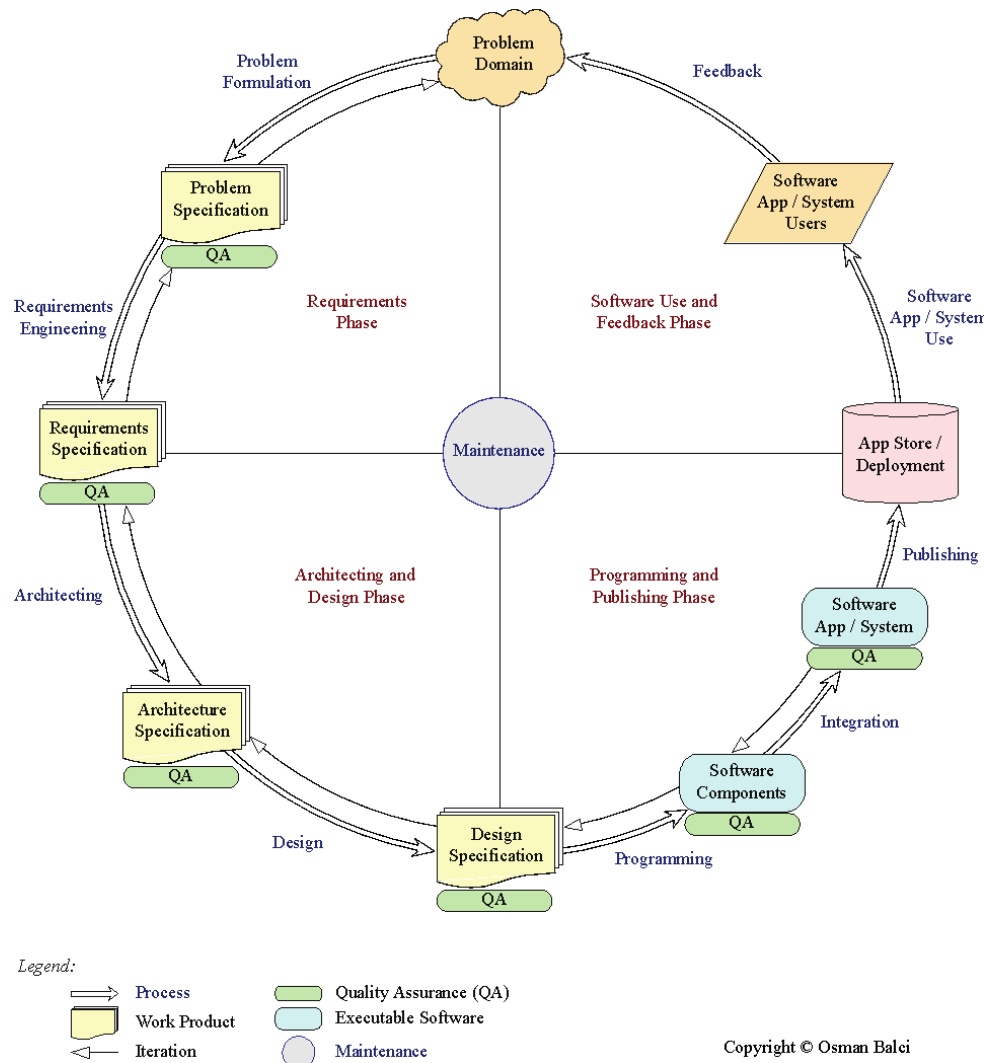
We developed this project using an AGILE methodology, and the team implemented all our planned features including authentication, dynamic UI components, and multiple API integration, while delivering additional features such as an AI chat bot and PDF printing as well. Feather Finder combines these functionalities with a user-friendly interface alongside unique styling and our own bird-themed color palette, offering a comprehensive package to combine many birdwatching resources into one application. This project was done collaboratively using GitHub, so we used effective teamwork, iterative development, and applied the use of different cloud software principles, resulting in a fully functional web application deployed on an Amazon EC2 server.

TABLE OF CONTENTS

USER ACCOUNTS.....	2
EXECUTIVE SUMMARY.....	3
1. SOFTWARE LIFE CYCLE.....	5
2. OUR OBJECTIVE.....	5
3. PROBLEM SPECIFICATION.....	6
3.1 WHAT IS THE PROBLEM?.....	6
3.2 EXPECTED CLOUD SOFTWARE-BASED SOLUTION.....	6
3.3 LEARNED AND NEW FEATURES PLANNED TO SOLVE THE PROBLEM.....	6
4. REQUIREMENTS SPECIFICATION.....	8
4.1 FUNCTIONAL REQUIREMENTS.....	8
4.2 NON-FUNCTIONAL REQUIREMENTS.....	8
5. ARCHITECTURE SPECIFICATION.....	9
6. DESIGN SPECIFICATION.....	10
6.1 USER INTERFACE DESIGN SPECIFICATION.....	10
6.2 DATABASE DESIGN SPECIFICATION.....	10
6.3 API DATA MODEL DESIGN SPECIFICATION.....	10
7. DELIVERED SOFTWARE FUNCTIONALITY.....	11
7.1 IMPLEMENTED LEARNED FEATURES.....	11
7.2 IMPLEMENTED NEW FEATURES.....	11
7.3 FUNCTIONALITY DESCRIPTION.....	12
8. CONCLUSIONS.....	12
9. SUBMISSION INSTRUCTIONS.....	13
10. PERCENTAGES OF CONTRIBUTION.....	14
11. CALCULATION OF GRADES.....	15
12. TEAM MEMBER RULES.....	15
13. GRADING SHEET.....	16
REFERENCES.....	17
APPENDIX A: MEETING MINUTES.....	18

1. SOFTWARE LIFE CYCLE

A good software engineer develops software by following the software life cycle shown below.



A programmer (hacker or ad-hoc developer) develops software by looking at the problem and directly coding in an IDE. This approach is known as the *Build-and-Fix Approach*, which must never be used!

2. OUR OBJECTIVE

The objective of our team project is to **demonstrate how capable the team members are** in engineering a Jakarta EE cloud software application to solve a complex problem. The cloud software application is created for the purpose of showing how *learned* and *new* complex functionalities and features the team members are capable of developing.

3. PROBLEM SPECIFICATION

3.1 What is the Problem?

The problem we aim to solve is the lack of accessible tools for bird-watchers and hikers to identify and document bird species they encounter on their daily travels. Bird enthusiasts may rely on physical guides and fragmented information online, which makes it difficult to track and identify information about birds in real time. Our solution is to provide a web application that allows users to identify birds in their surrounding, and by incorporating the eBird and Nuthatch API into our application, alongside various other features, we can allow the user to log their sightings, store the count of different species of birds they've seen, and potentially ask a Bird Chatbot questions about different bird species. Alongside this, we will incorporate the Google Map API, to allow the user to see a real-time view of the birds in their surroundings, enhancing interactivity and allowing for faster identification of birds. The web application also allows users a way to compete with other birdwatchers, tracking how many species they've seen, and listing the users in a leaderboard to see who has the highest score.

3.2 Expected Cloud Software-Based Solution

1. Identify the APIs you plan to use.
 - a. eBird API
 - i. API that returns JSON data of birds seen recently in a general given area. The returned information includes: The species (scientific and common name), How many were seen, and the Longitude and Latitude of where it was seen.
 - ii. <https://documenter.getpostman.com/view/664302/S1ENwy59>
 - b. nuthatch API
 - i. API that returns additional general information for birds, including species information, wingspan, endangerment status, and images.
 - ii. <https://nuthatch.lastelm.software/swagger.html>
 - c. Gemini API (Gemini 1.5 Flash)
 - i. API that processes user requests/input and generates an appropriate response to address the request
 - ii. <https://ai.google.dev/>
 - d. Google Maps API
 - i. API that will give the location of the bird that was found and the route of where it was found.
 - ii. https://developers.google.com/maps/documentation/embed/get-started?hl=en&_gl=1*mygnja*_ga*MjU1MzIxMTA3LjE3MzAwNjY2NjI.*_ga_NRWSTWS78N*MTczMDA2NjY2MS4xLjEuMTczMDA2Njc2My4wLjAuMA..
2. Imagine the completed app to solve the problem and describe its overall functionality.
 - a. Birds Seen database
 - b. No API data will be fetched here
 - c. This database will hold a foreign key for a user id, so that each database of birds seen will be associated with a specific user.

- i. Each user will also hold data for number of birds seen, so that there can be a leaderboard system

3.3 Learned and New Features Planned to Solve the Problem

The software application we plan to develop is expected to implement the following learned features to be able to solve the problem stated above:

	Topic	Learned Feature
☑	API (Required)	1. Enable the user to enter a search query, 2. Obtain complex JSON data from an API for the search query, 3. Process the JSON data safely by using Optionals, and 4. Display results using the data extracted from the JSON data.
☑	MySQL Database (Required)	1. Create a SQL file containing example content with minimum 10 items, 2. Create the MySQL database with many entities and their relationships, 3. Populate the database by running the SQL file, 4. Enable the user to perform CRUD operations on the database, and 5. Enable the user to perform complex searches on the database.
☑	User Account (Required)	1. User account creation and maintenance, 2. User profile photo upload or take by using web camera, 3. Authentication with username and password, 4. Password reset by answering a security question, and 5. Salting and Hashing for secure storage and retrieval of a password with Key Stretching.
☑	User Interface (Required)	1. Create a template-based UI with header and footer, and 2. Enable the user to navigate through the app easily.
☑	Deployment (Required)	1. Deploy your app to run on your Amazon EC2 virtual private server, and 2. Enable the use of your app by anyone anywhere and anytime.

☐	Accordion Panel	Display a container component with vertically stacked expandable / collapsable panels	BevQ
☐	Chart	Create and Display a Bar Chart	Countries
☐	Content Flow	Use PrimeFaces Content Flow to display flows of photos or images	Countries Movies
☐	Create Your Own API	Create your own RESTful API by transforming a database into Web Services	VTBuildingsAPI
☐	Data Table	Sortable per column and searchable globally or with respect to a column	Countries Movies
☐	Data Table Export	Export a Data Table's all data or shown-page data as an Excel, PDF, CSV, or XML file.	Companies BevQ
☐	Email	Compose and send an email message with multimedia content	Fitness
☐	File Download	Download a file from a directory external to the app's deployment directory	CloudDrive
☐	File Upload	Upload a file, store it in, and retrieve it from a directory external to the app's deployment directory	CloudDrive Companies
☒	Image Slider	Display an image slider of photos or images	Fitness Countries
☐	Internationalization	Display the user interface in multiple languages	PizzaHut
☒	Map	Display a standard, satellite or terrain Google map by using Google Maps JavaScript API	VTQuest
☐	Map Directions	Display walking, driving, bicycling or transit directions using the Google Maps Directions API	VTQuest
☐	Map Location	Mark a location with an annotation pin on a standard or satellite Google map	VTQuest
☐	Map Pin	Show a location's name above its annotation pin by clicking the pin	VTQuest
☐	Menu – Drop Down	Display a navigation component with one level of submenus	Most tutorial apps
☐	Menu – Sidebar	Provide a menu in a sidebar that is a panel component displayed as an overlay	CloudDrive BevQ PizzaHut
☒	Menu – Tiered	Display submenus in nested overlays	Countries
☐	Pick List	Create a Pick List with Source and Target and enable drag and drop transfer and reordering	Countries
☒	Print or Save as PDF	Print a designated part of a JSF page or save it as PDF	Fitness
☐	Radio	Stream and play a live radio broadcast with Play and Pause controls	Fitness
☒	reCAPTCHA	Implement reCAPTCHA security service that protects your app from fraud and abuse	CloudDrive
☐	Text Message	Compose and send a text message to a mobile phone in plain text format	Fitness
☐	Video Player – YouTube	Display the YouTube video player in a <i>Responsive</i> manner with its controls	Fitness

We also plan to implement the following new features to provide an enhanced solution to the problem stated above.

1. AI Chatbot so users can ask questions to it and get answers

4. REQUIREMENTS SPECIFICATION

This section specifies the Functional and Non-Functional Requirements under which our Jakarta/Java EE cloud software application will be developed.

4.1 Functional Requirements

1. The user of the software system shall be able to authenticate with username and password.
2. Username shall consist of minimum of 6 and maximum of 12 alphabet characters. Passwords shall consist of minimum 8 and maximum 16 characters of which at least 2 must be uppercase letters, at least 2 must numbers and at least 2 must be special characters.
3. The user shall be able to input phone or email for two factor authentication of login.
4. The user shall be able to edit their profile information, such as name and location.
5. The user shall be able to upload their own profile picture to the application.
6. The user shall be able to access their camera in order to take a photo.
7. The application shall be able to print or download a bird photo as a pdf.
8. The user shall be able to view the sightings list of the bird they have seen and save them in their account.

9. The user shall be able to create, edit, view, delete, google maps location of the bird sightings in the database.
10. The user shall be able to find specific birds in the database by searching with some condition specified by the user.
11. The user shall be able to search a specific region using birds APIs to find birds in the regions.
12. The user shall be able to add a sighting from the API search into their personal sightings database.
13. The user shall be able to view a map of where the last sighting was of a bird in order to possibly view the same birds.
14. The user shall be able to see the image slider, and the header name (with their first name to indicate logged in) in the home page of the application.
15. The users shall be able to check reCAPTCHA before signing in their account.
16. The user shall be able to look at the photo carousel of different birds and their information.
17. The user shall be able to ask the AI chatbot about general questions regarding birds.
18. The user shall be able to ask the AI chatbot about a selected bird's diet.
19. The user shall be able to ask the AI chatbot about a selected bird's fun facts.
20. The user shall be able to ask the AI chatbot about a selected bird's predators.
21. The user shall be able to play, pause, and stop the application at any time during the course of software execution.
22. The user shall be able to see the number of birds seen in the leaderboard function of their account.

4.2 Non-Functional Requirements

1. The UI template footer shall contain the following statement:

[CS3754 Cloud Software Development](#) course semester project application developed by studentName1, studentName2, studentName3, studentName4, and studentName5.

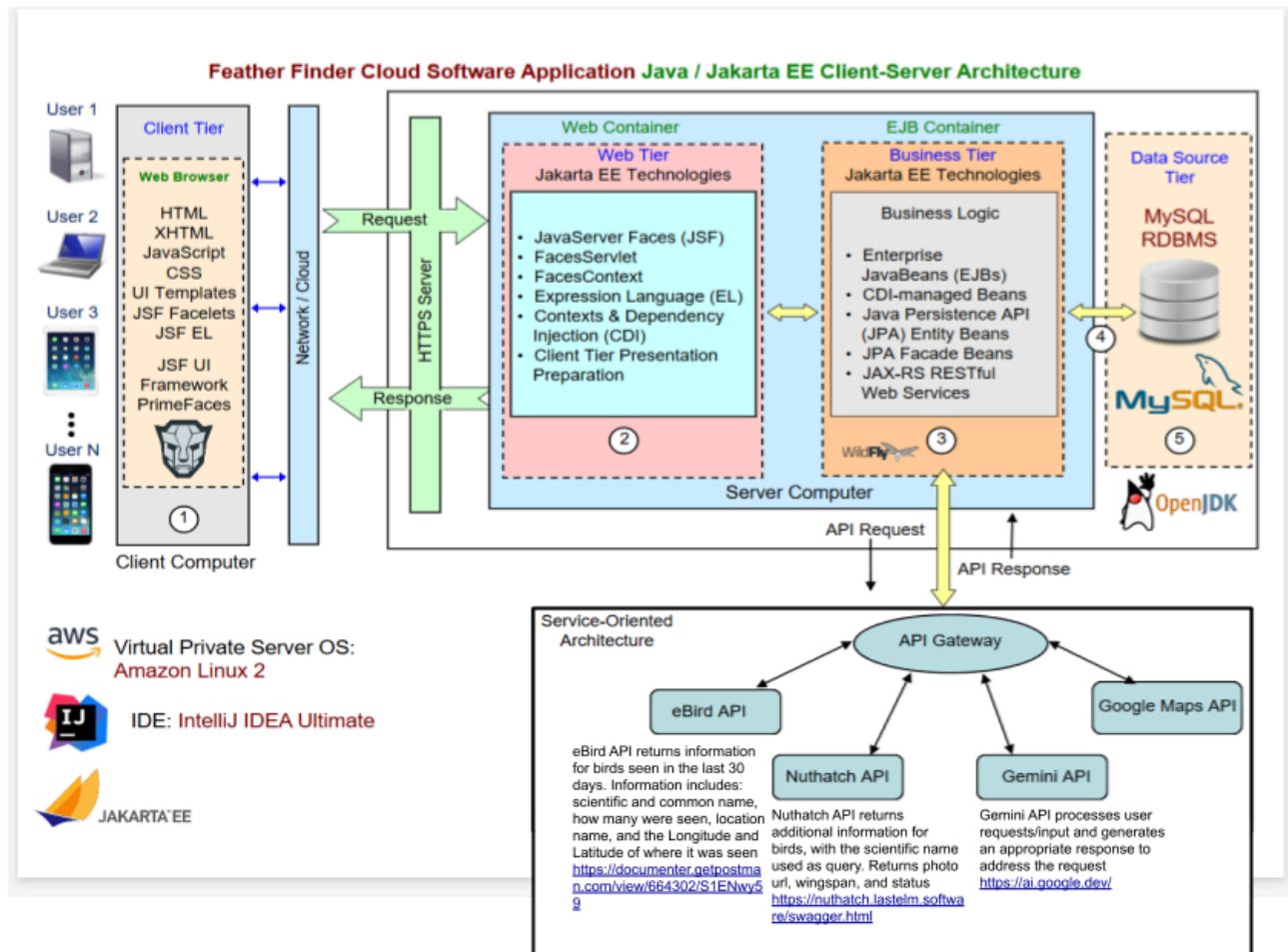
Course title and each student name above shall be hyperlinked to show the corresponding homepage *in a new window / tab*. Use VT website if you do not have one for yourself.

2. The cloud software application name shall be a meaningful name without course number or team number. The app name shall reflect what the app does such as MeetingScheduler.
3. The application shall load key functions within 5 seconds to ensure a responsive and smooth user experience.
4. All software documentation shall be provided in HTML-based hypertext format.
5. All software input data shall be specified in XML format.
6. The software system shall be developed using royalty-free software products.
7. The app shall have a clear and easy-to-navigate design so new users can understand and interact within their first session.
8. The app shall be fully functional on web browsers.
9. The app shall be accessible via mobile browsers on iOS and Android devices.
10. The app shall limit data collection to only necessary user information.
11. The app shall include basic accessibility features to ensure it is usable to any users.
12. The application shall be deployed in the Amazon EC2 server.
13. The application shall use the MySQL database

14. The app shall use eBird API and nuthatch API.
15. The app shall use google Maps API and Gemini API

5. ARCHITECTURE SPECIFICATION

1. eBird API
 - a. API that returns JSON data of birds seen recently in a general given area. The returned information includes: The species (scientific and common name), How many were seen, and the Longitude and Latitude of where it was seen. This API returns information for birds seen in the last 30 days. Some locations may not have any data, and some may have large amounts.
 - b. <https://documenter.getpostman.com/view/664302/S1ENwy59>
2. nuthatch API
 - a. API that returns additional general information for birds, including species information, wingspan, endangerment status, and images.
 - b. <https://nuthatch.lastelm.software/swagger.html>
3. Gemini API (Gemini 1.5 Flash)
 - a. API that processes user requests/input and generates an appropriate response to address the request
 - b. <https://ai.google.dev/>
4. Google Maps API
 - a. API that will give the location of the bird that was found and the route of where it was found.
 - b. https://developers.google.com/maps/documentation/embed/get-started?hl=en&_gl=1*_mygnja*_ga*MjU1MzkxMTA3LjE3MzAwNjY2NjI.*_ga_NRWSTWS78N*MTczMDA2NjY2MS4xLjEuMTczMDA2Njc2My4wLjAuMA..

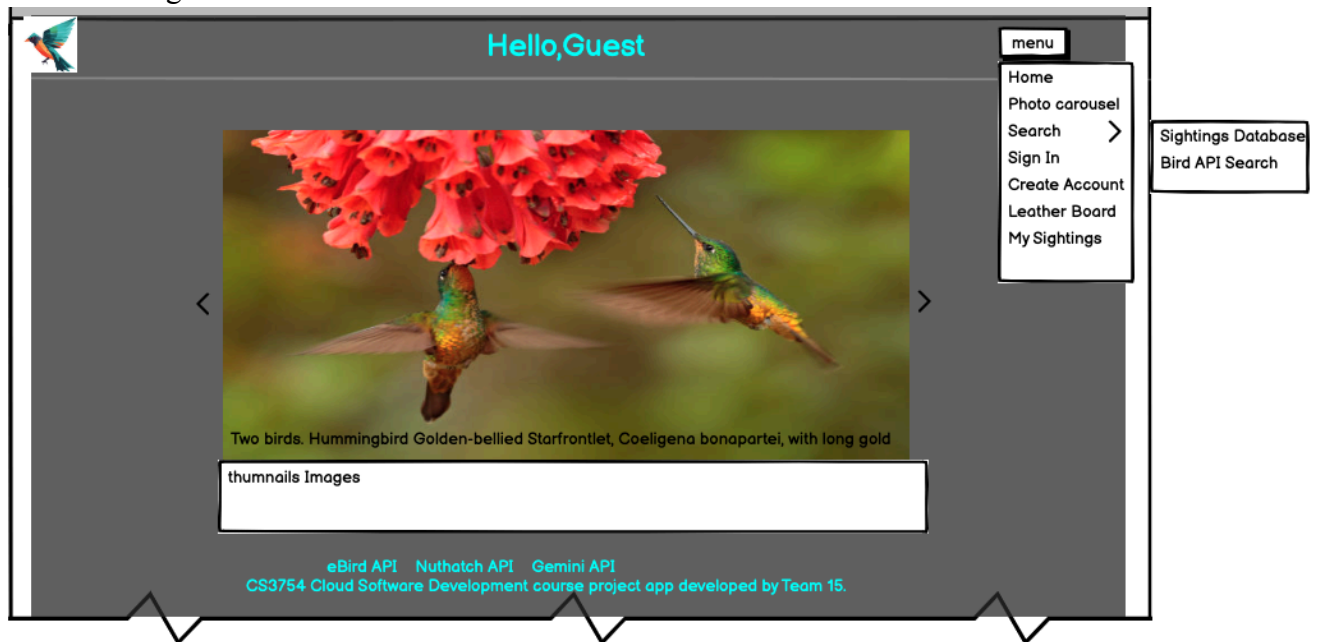


6. DESIGN SPECIFICATION

6.1 User Interface Design Specification

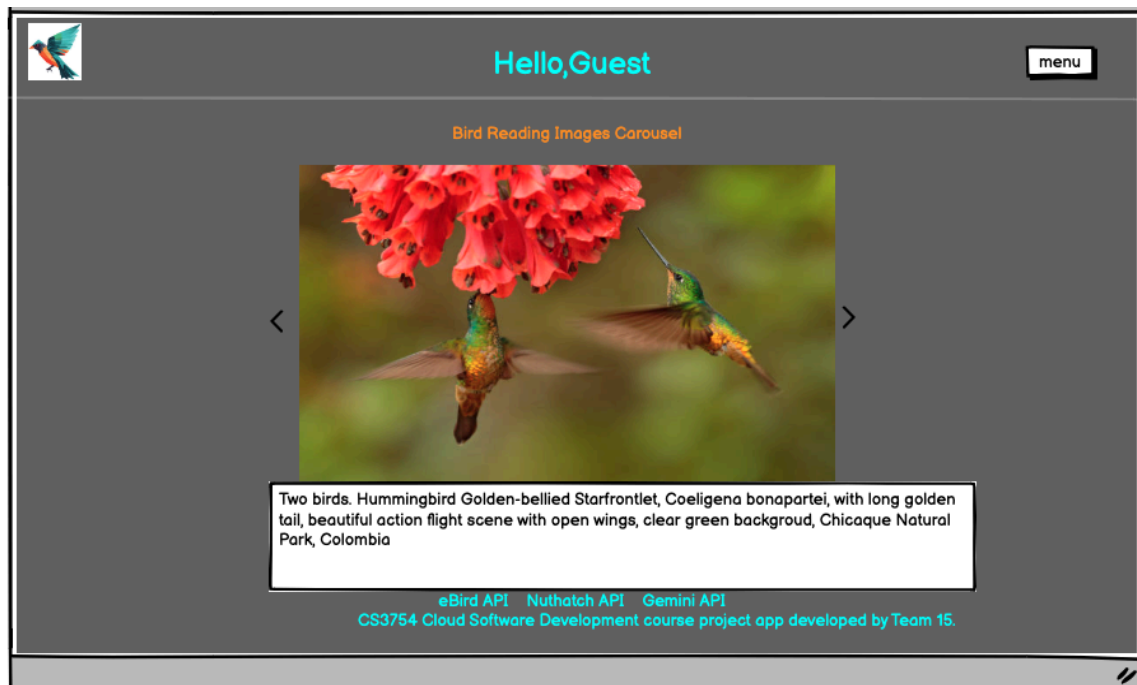
For our app, we use the website [Balsamiq.com](https://balsamiq.com) to make our UI Wireframes. Here is the link of the wireframes we did: <https://balsamiq.cloud/sis310f/pur215c/r18E1> to see it more clearly. The name of the app is Feather Finder.

6.1.1 Home Page



6.1.2 Photo Carousel Function

This function will show the image of a bird with description to make users look the picture before they discover other functions in the app if they want to.



6.1.3 Sighting Database Search

We create the database search with some conditions below and the UI wireframes of each search type, and result database UI Wireframe


Hello,Guest
menu

Sighting Database Search



Type	Description	Search
1	Search Sighting Name, location or how many sighting individual or Search of it	Q Search Type 1
2	Sighting name contains NameQ AND numberOfSightings >= numberOfSightingsQ	Q Search Type 2
3	Sighting Location contains locationQ AND numberOfSightings >= numberOfSightingsQ	Q Search Type 3
4	Sighting name contains nameQ AND Sighting location contains locationQ	Q Search Type 4

Search Type 1

Search Category

Enter Search String

Submit

Cancel

Search Type 2

Sighting Name

Sighting numberOfSighting>=

Submit

Cancel

Search Type 3

Sighting Location

Sighting numberOfSighting<=

Submit

Cancel

Search Type 4

Sighting Name

Sighting Location

Submit

Cancel

eBird API Nuthatch API Gemini API

CS3754 Cloud Software Development course project app developed by Team 15.


Hello,Guest
menu

Sighting Search Results

<<

1

2

3

4

>>

10

Common Name	Scientific Name	Image	Location	Observation Notes	Number of Sightings
Rock Pigeon	Columbia livia		piazza San Piero	Birds	3
American Coot	Fulica Americana		Fall Mills Lake	A Duck on the water	12

View

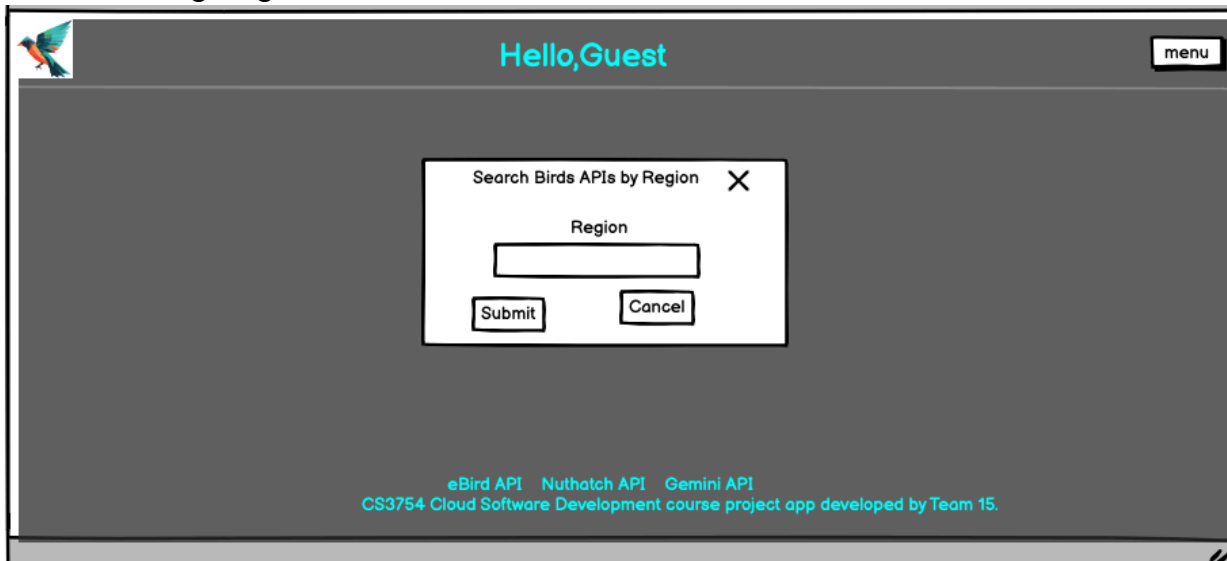
Map

Go Back

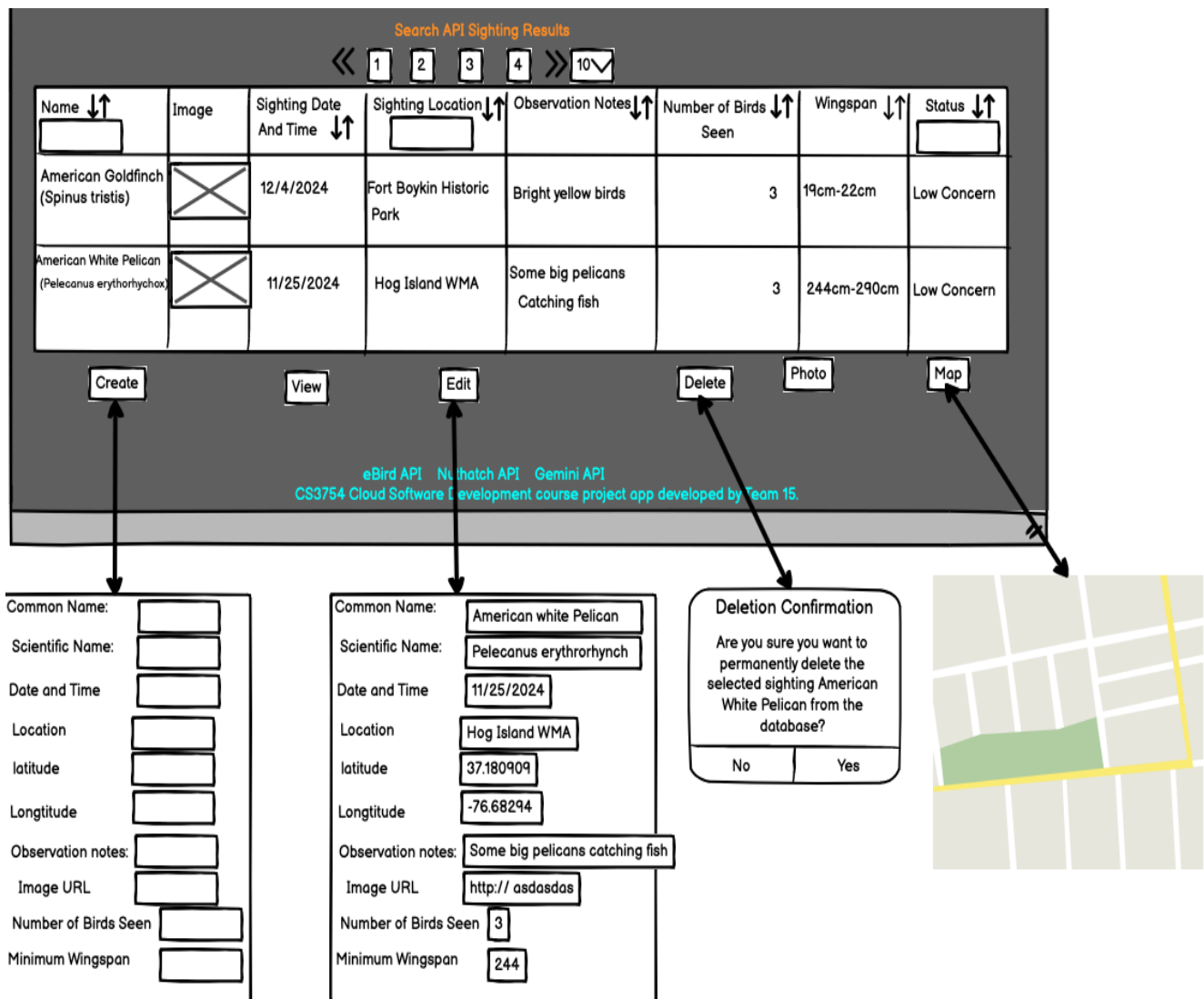
eBird API Nuthatch API Gemini API

CS3754 Cloud Software Development course project app developed by Team 15.

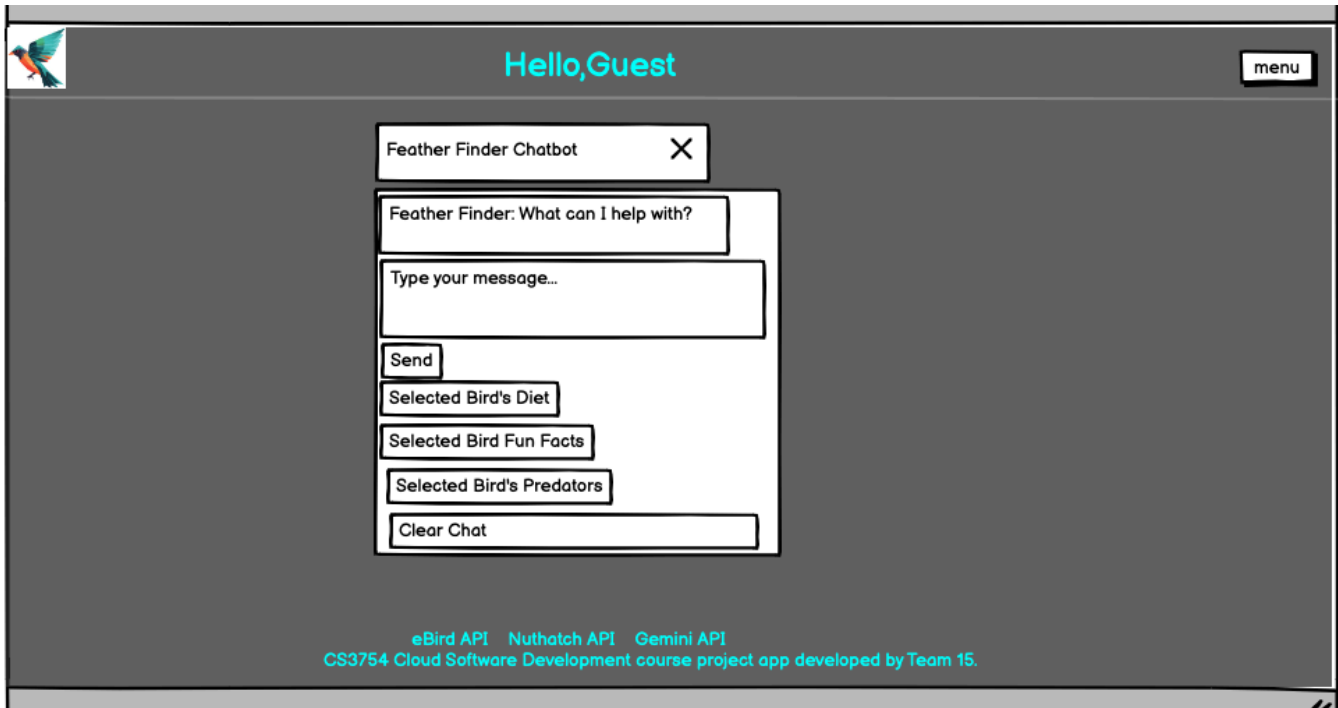
6.1.4 Sighting Bird APIs Search UI Wireframe



The APIs Results UI wireframe

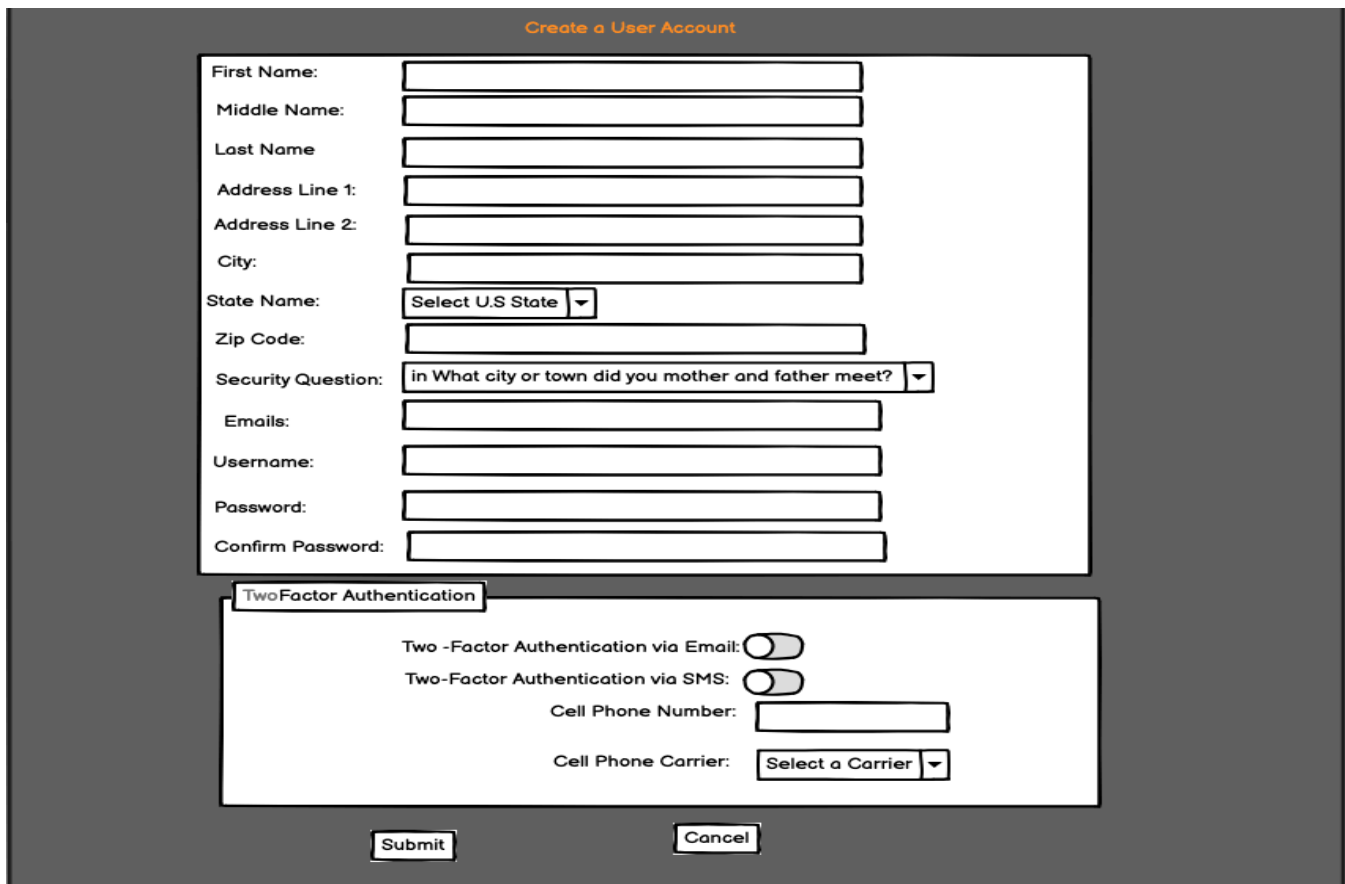


6.1.5 AI Chatbot UI Wireframe



The wireframe shows a chatbot interface titled "Hello, Guest" with a "menu" button. The chatbot is named "Feather Finder Chatbot" and has a close button (X). The chat area contains the message "Feather Finder: What can I help with?" and a text input field with the placeholder "Type your message...". Below the input field is a "Send" button. The chatbot's response area includes four buttons: "Selected Bird's Diet", "Selected Bird Fun Facts", "Selected Bird's Predators", and "Clear Chat". At the bottom, there is a footer with the text "eBird API Nuthatch API Gemini API" and "CS3754 Cloud Software Development course project app developed by Team 15."

6.1.6 Create Account UI Wireframe



The wireframe shows a "Create a User Account" form. The form includes the following fields and controls:

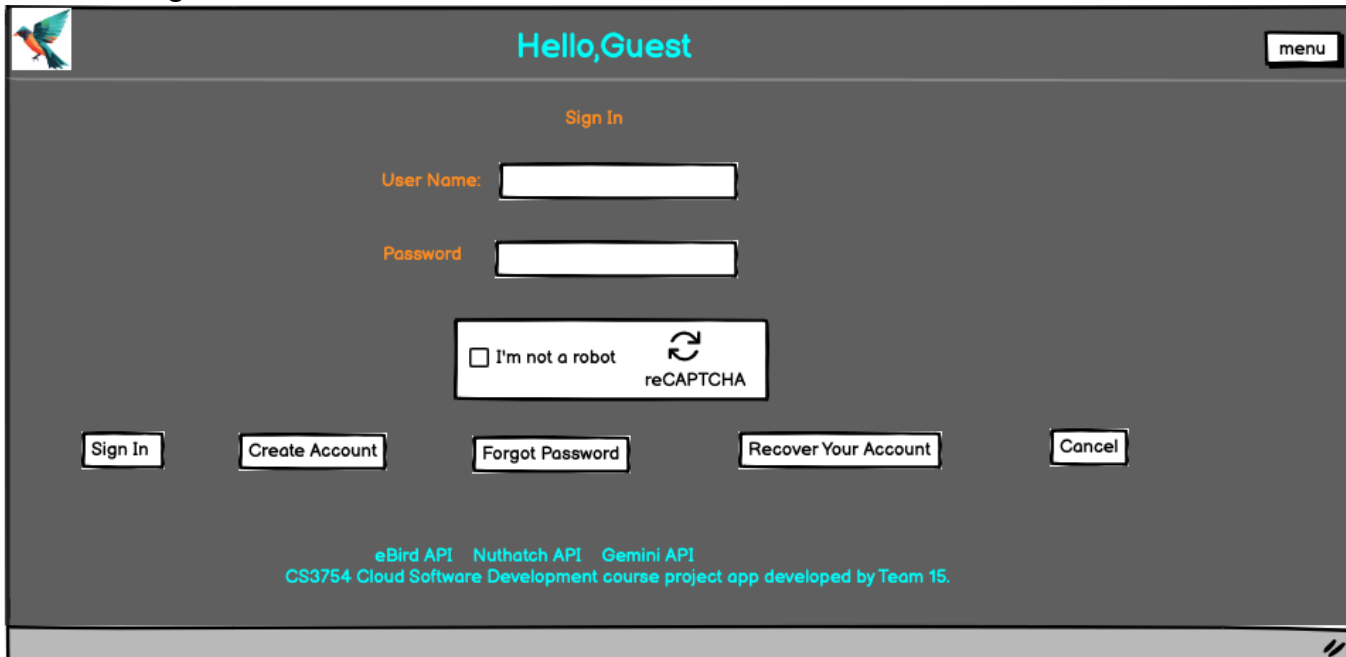
- First Name:
- Middle Name:
- Last Name:
- Address Line 1:
- Address Line 2:
- City:
- State Name:
- Zip Code:
- Security Question:
- Emails:
- Username:
- Password:
- Confirm Password:

Below the main form is a "TwoFactor Authentication" section with the following controls:

- Two -Factor Authentication via Email: ☐
- Two-Factor Authentication via SMS: ☐
- Cell Phone Number:
- Cell Phone Carrier:

At the bottom of the form are two buttons: "Submit" and "Cancel".

6.1.7 SignIn UI Wireframe



6.1.7 SignIn UI Wireframe

The wireframe shows a sign-in interface with a dark gray background. At the top left is a small bird icon. The header "Hello, Guest" is in teal, and a "menu" button is at the top right. The "Sign In" text is centered in orange. Below it are input fields for "User Name:" and "Password:". A reCAPTCHA widget is centered, containing a checkbox for "I'm not a robot" and a "reCAPTCHA" logo. At the bottom are five buttons: "Sign In", "Create Account", "Forgot Password", "Recover Your Account", and "Cancel". The footer contains API links and project information in teal.

Sign In

User Name:

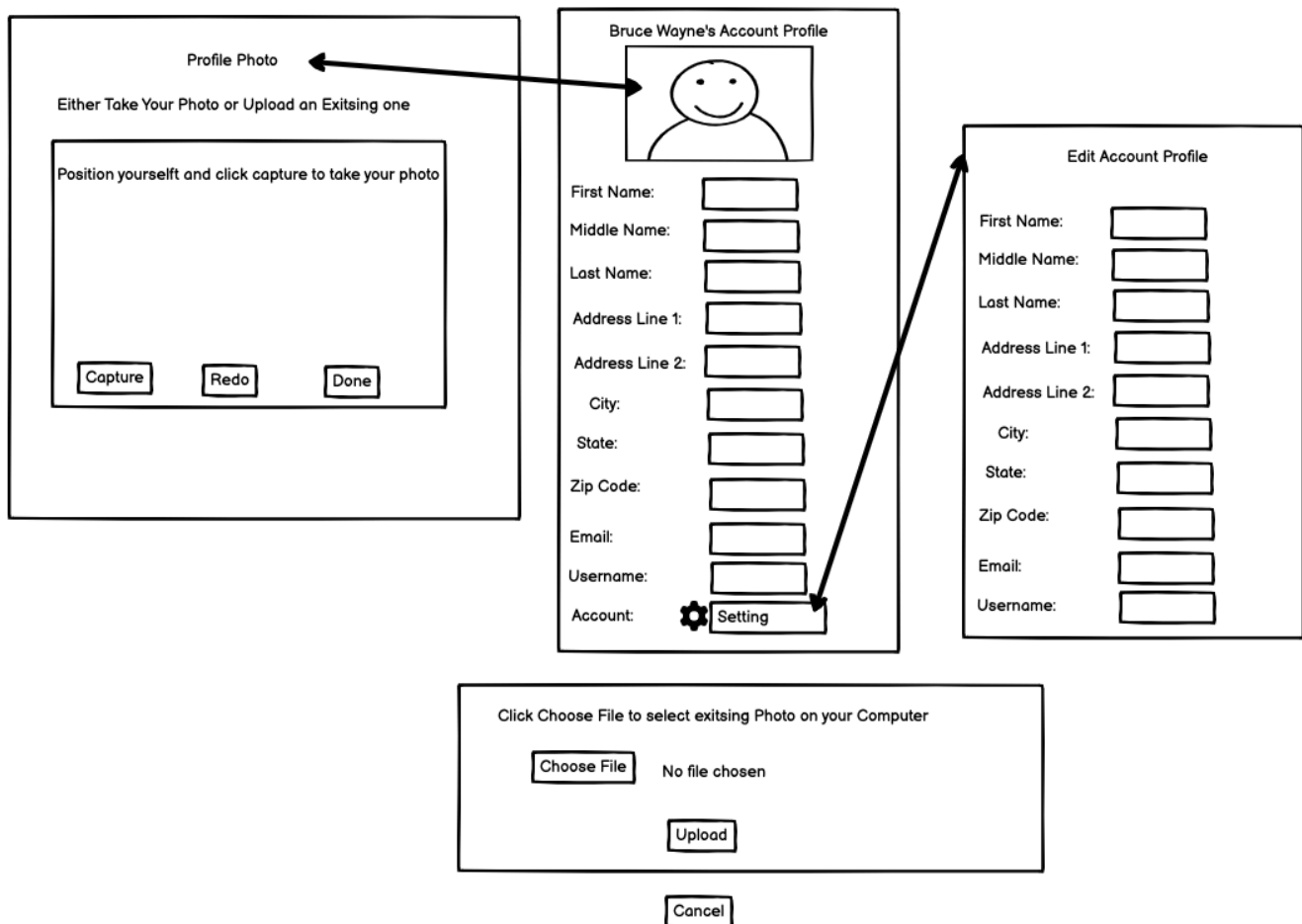
Password:

☐ I'm not a robot  reCAPTCHA

Sign In Create Account Forgot Password Recover Your Account Cancel

eBird API Nuthatch API Gemini API
CS3754 Cloud Software Development course project app developed by Team 15.

6.1.8 Account Profile, Edit Profile and Photo Profile UI Wireframe



6.1.8 Account Profile, Edit Profile and Photo Profile UI Wireframe

The wireframe shows three main components: a photo capture interface, a profile view, and an edit profile form.

Profile Photo: A box titled "Profile Photo" with the instruction "Either Take Your Photo or Upload an Existing one". It contains a large "Position yourself and click capture to take your photo" area with "Capture", "Redo", and "Done" buttons.

Bruce Wayne's Account Profile: A vertical list of fields: First Name, Middle Name, Last Name, Address Line 1, Address Line 2, City, State, Zip Code, Email, Username, and Account. Each field has a corresponding input box. A "Setting" button with a gear icon is at the bottom.

Edit Account Profile: A form with the same fields as the profile view, each with an input box.

File Selection: A box titled "Click Choose File to select existing Photo on your Computer" with a "Choose File" button, "No file chosen" text, an "Upload" button, and a "Cancel" button.

Profile Photo

Either Take Your Photo or Upload an Existing one

Position yourself and click capture to take your photo

Capture Redo Done

Bruce Wayne's Account Profile

First Name:

Middle Name:

Last Name:

Address Line 1:

Address Line 2:

City:

State:

Zip Code:

Email:

Username:

Account:  Setting

Edit Account Profile

First Name:

Middle Name:

Last Name:

Address Line 1:

Address Line 2:

City:

State:

Zip Code:

Email:

Username:

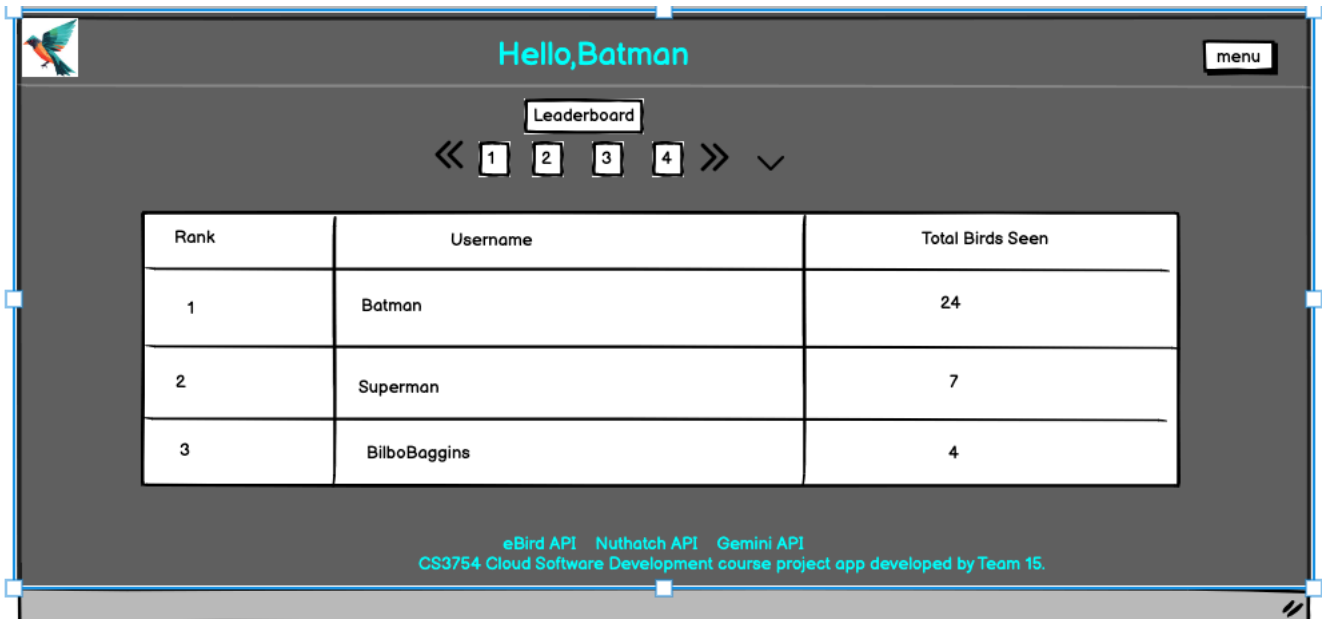
Click Choose File to select existing Photo on your Computer

Choose File No file chosen

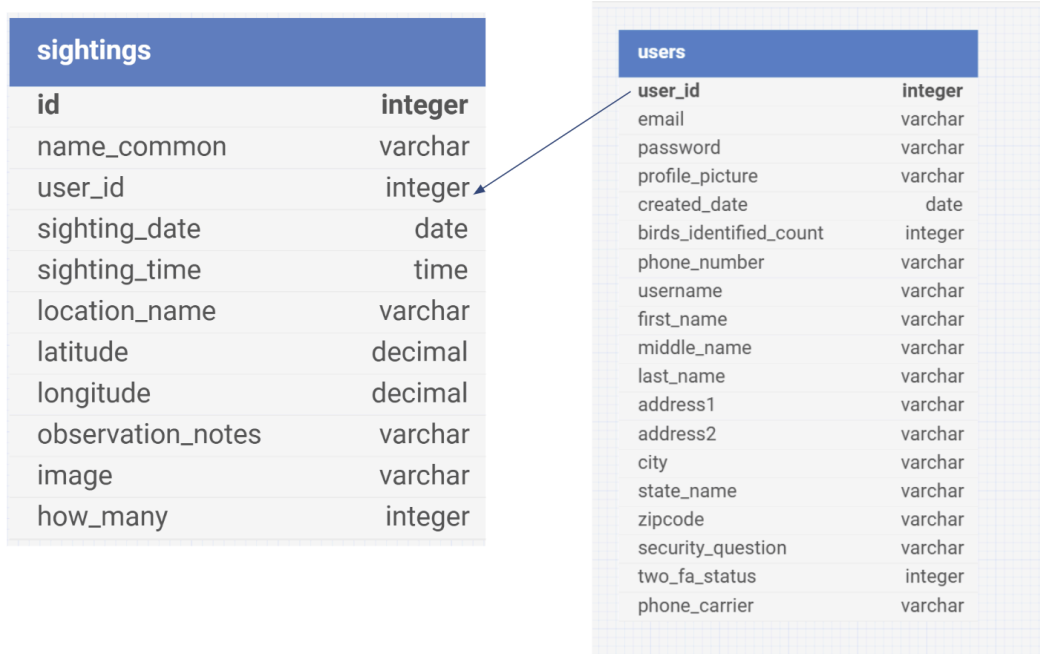
Upload

Cancel

6.1.9 Leaderboard UI Writeframe



6.2 Database Design Specification



For our web application, we have two primary tables: sightings and users. The sightings table stores information about each bird sighting and includes fields such as the bird's common name, the ID of the user who recorded the sighting (`user_id`) where `user_id` is a foreign key from the users table, the data and time of the sighting (`sighting_date`, `sighting_time`), the location details (`location_name`, latitude and longitude), observation notes, and the number of birds observed. The users table tracks all of the users information specifically for creating an account and it has an extra field, `birds_identified_count` to keep track of the individual sightings each user has, a feature useful for implementing our leaderboard.

6.3 API Data Model Design Specification

eBird API:

```
{
  "speciesCode": "rocpig",
  "comName": "Rock Pigeon",
  "sciName": "Columba livia",
  "locId": "L10233102",
  "locName": "Ahmet Yesevi University",
  "obsDt": "2020-01-20 13:03",
  "howMany": 40,
  "lat": 43.2995121,
  "lng": 68.3065226,
  "obsValid": true,
  "obsReviewed": false,
  "locationPrivate": false,
  "subId": "S63579583"
},
```

nuthatch API:

```
{
  "entities": [
    {
      "images": [
        "https://images.unsplash.com/photo-1600892457290-84afbbe878f0",
        "https://images.unsplash.com/photo-1604605152447-1fcea1a333f3"
      ],
      "lengthMin": "32",
      "lengthMax": "40",
      "name": "Barn Owl",
      "wingspanMin": "100",
      "id": "295",
      "wingspanMax": "125",
      "sciName": "Tyto alba",
      "region": [
        "North America"
      ],
      "family": "Tytonidae",
      "order": "Strigiformes",
      "status": "Low Concern"
    }
  ],
  "total": 1,
  "page": 1,
  "pageSize": 25
}
```

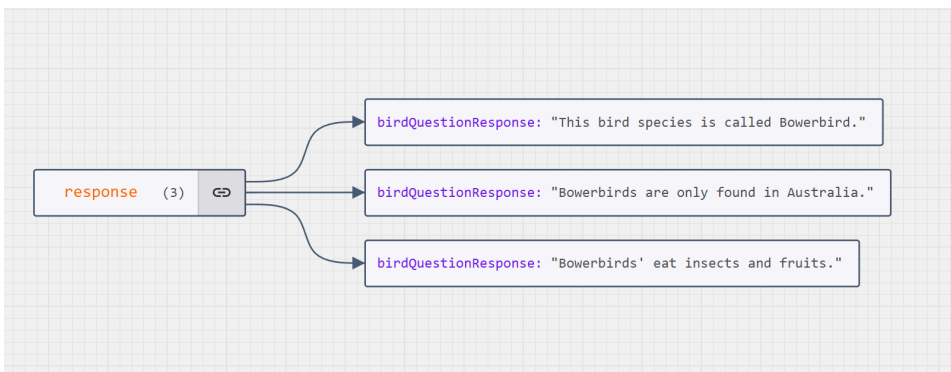
Gemini API:

```
{
  "response": [
    {
      "birdQuestionResponse": "This bird species is called Bowerbird."
    },
    {
      "birdQuestionResponse": "Bowerbirds are only found in Australia."
    },
    {
      "birdQuestionResponse": "Bowerbirds' eat insects and fruits."
    }
  ]
}
```

question: "What species of bird am I looking at?"

question: "Where is this bird typically found?"

question: "What kind of food does this bird eat?"



7. DELIVERED SOFTWARE FUNCTIONALITY

>>> Grading Notes:

- (-5) App name contains Team or meaningless
- (-15) No MySQL database used for app data storage and retrieval
- (-15) No MySQL database complex searches implemented properly

- (-15) No API is used with search query entered by the user
- (-75) Not running on Amazon EC2 VPS. The submission is rejected. 25 pts given for the effort.
- (-10) Empty list (e.g., favorites) with no example content of at least 10 items
- (-15) No User Account creation and maintenance
- (-10) No template-based UI with header and footer
- (-10) Number of project team meeting minutes < 5 and/or their content lacks sufficient detail
- (-X) Other missing features typically implemented in tutorials

<<<

Our delivered software application implements the following features:

7.1 Implemented Learned Features

Our delivered software application implements the following learned features:

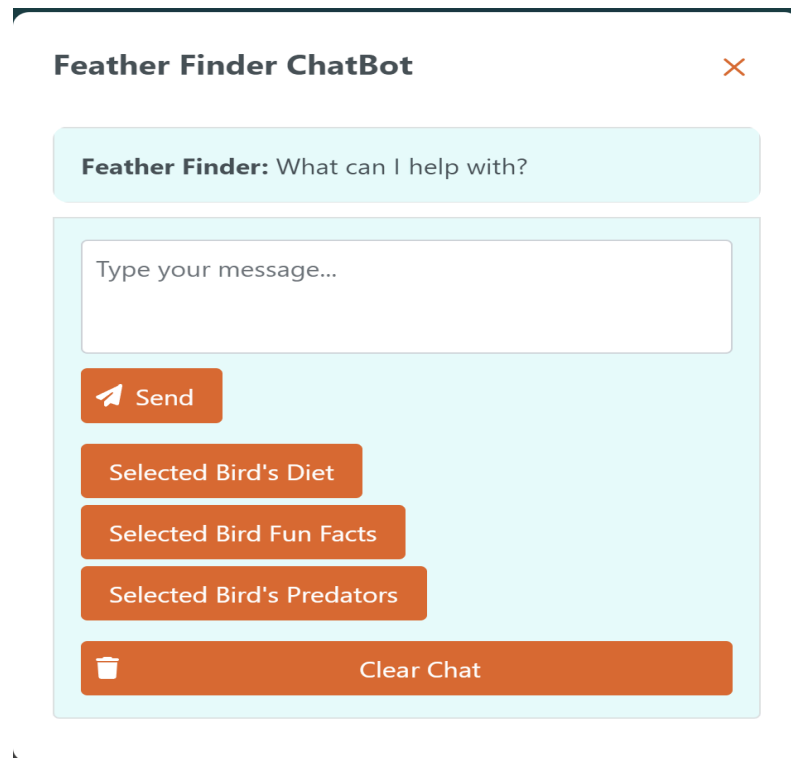
☒	API (Required)	1. Enable the user to enter a search query, 2. Obtain complex JSON data from an API for the search query, 3. Process the JSON data safely by using Optionals, and 4. Display results using the data extracted from the JSON data.
☒	MySQL Database (Required)	1. Create a SQL file containing example content with minimum 10 items, 2. Create the MySQL database with many entities and their relationships, 3. Populate the database by running the SQL file, 4. Enable the user to perform CRUD operations on the database, and 5. Enable the user to perform complex searches on the database.
☒	User Account (Required)	1. User account creation and maintenance, 2. User profile photo upload or take by using web camera, 3. Authentication with username and password, 4. Password reset by answering a security question, and 5. Salting and Hashing for secure storage and retrieval of a password with Key Stretching.
☒	User Interface (Required)	1. Create a template-based UI with header and footer, and 2. Enable the user to navigate through the app easily.
☒	Deployment (Required)	1. Deploy your app to run on your Amazon EC2 virtual private server, and 2. Enable the use of your app by anyone anywhere and anytime.

☐	Accordion Panel	Display a container component with vertically stacked expandable / collapsable panels	BevQ
☐	Chart	Create and Display a Bar Chart	Countries
☐	Content Flow	Use PrimeFaces Content Flow to display flows of photos or images	Countries Movies
☐	Create Your Own API	Create your own RESTful API by transforming a database into Web Services	VTBuildingsAPI
☐	Data Table	Sortable per column and searchable globally or with respect to a column	Countries Movies
☐	Data Table Export	Export a Data Table's all data or shown-page data as an Excel, PDF, CSV, or XML file.	Companies BevQ
☐	Email	Compose and send an email message with multimedia content	Fitness
☐	File Download	Download a file from a directory external to the app's deployment directory	CloudDrive
☐	File Upload	Upload a file, store it in, and retrieve it from a directory external to the app's deployment directory	CloudDrive Companies
☒	Image Slider	Display an image slider of photos or images	Fitness Countries
☐	Internationalization	Display the user interface in multiple languages	PizzaHut
☒	Map	Display a standard, satellite or terrain Google map by using Google Maps JavaScript API	VTQuest
☐	Map Directions	Display walking, driving, bicycling or transit directions using the Google Maps Directions API	VTQuest
☐	Map Location	Mark a location with an annotation pin on a standard or satellite Google map	VTQuest
☐	Map Pin	Show a location's name above its annotation pin by clicking the pin	VTQuest
☐	Menu – Drop Down	Display a navigation component with one level of submenus	Most tutorial apps
☐	Menu – Sidebar	Provide a menu in a sidebar that is a panel component displayed as an overlay	CloudDrive BevQ PizzaHut
☒	Menu – Tiered	Display submenus in nested overlays	Countries
☐	Pick List	Create a Pick List with Source and Target and enable drag and drop transfer and reordering	Countries
☒	Print or Save as PDF	Print a designated part of a JSF page or save it as PDF	Fitness
☐	Radio	Stream and play a live radio broadcast with Play and Pause controls	Fitness
☒	reCAPTCHA	Implement reCAPTCHA security service that protects your app from fraud and abuse	CloudDrive
☐	Text Message	Compose and send a text message to a mobile phone in plain text format	Fitness
☐	Video Player – YouTube	Display the YouTube video player in a <i>Responsive</i> manner with its controls	Fitness

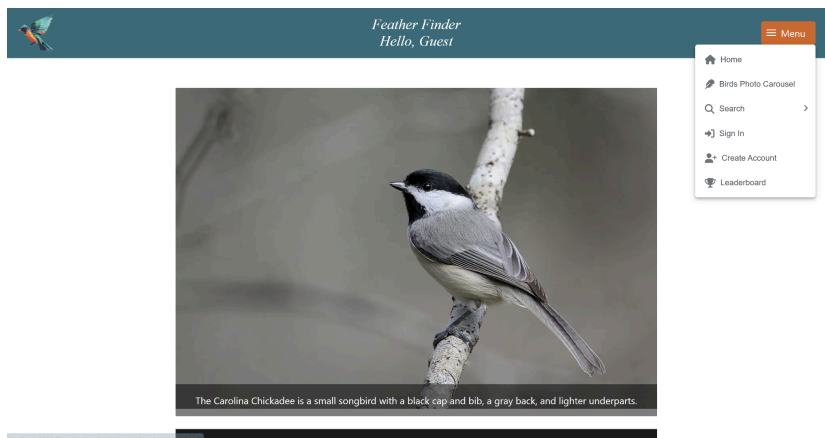
7.2 Implemented New features

Our delivered software application implements the following new features:

1. Bird Info Gemini AI Chatbot: We implemented a chat bot feature that answers users questions about different birds or a selected bird. Users can click on the Feather Finder Chat Bot button on the footer, and a dialog will open where users can ask open-ended questions relating to any species of birds, or select from a range of already created queries about a selected bird species diet, fun facts, and predators. This chat bot feature makes use of Google's Gemini API and we are currently using the free-tier.



7.3 Functionality Description



Home page of the web application. From here, users can see an image slider of various birds with associated facts. From here, the user can open the menu, which can take the user to the home page, a photo carousel page, allow searching of the birds apis, see the bird sightings leaderboard, and in the case of not logged in, a user can create an account or sign in. This is a tiered menu to account for the multiple types of searches you can do: api and database (database only appears when logged in)

If the user wishes to create an account: The user inputs all the required information indicated by a *. This includes information like name, address, security question, email, username, password. This is for the user account feature, allowing the user to sign in, upload a photo, and have authentication with username and password.

Create a User Account

First Name: *	
Middle Name:	
Last Name: *	
Address Line 1: *	
Address Line 2:	
City: *	
State Name:	Select U.S. State ▾
Zip Code: ? *	
Security Question:	In what city or town did your mother and father meet? ▾
Security Answer: *	
Email: *	
Username: ? *	
Password: ? *	
Confirm Password: *	

After inputting all information, the user can select two-factor authentication, which can be done via email or sms, which the user can select.

Two-Factor Authentication

Two-Factor Authentication via Email: ☐

Two-Factor Authentication via SMS: ☐

Cell Phone Number:

Cell Phone Carrier: Select a Carrier ▾

Once the account is created, the user can then sign in from the sign in page: The sign in page has the

user input their username and password, and then select the reCAPTCHA checkbox for extra security.

Sign In

Username: *

Password: *

☐ I'm not a robot


reCAPTCHA
[Privacy](#) - [Terms](#)

[Sign In](#) [Create Account](#) [Forgot Password](#) [Recover Your Account](#) [Cancel](#)

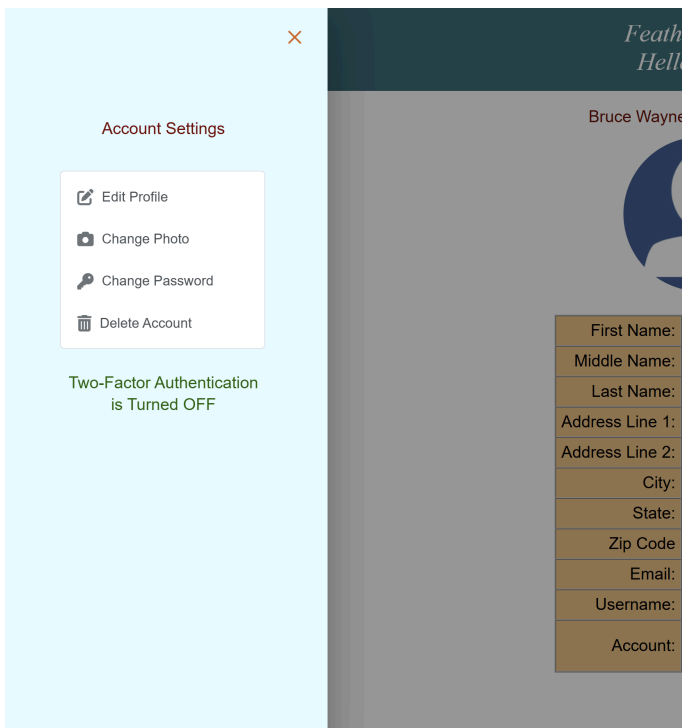
Upon sign in, the user is directed to their accounts page, which shows their user information.

Bruce Wayne's Account Profile



First Name:	Bruce
Middle Name:	
Last Name:	Wayne
Address Line 1:	12345 Wayne Manor
Address Line 2:	
City:	Gotham
State:	New York
Zip Code:	12345
Email:	batman@gmail.com
Username:	Batman
Account:	Settings

The user may update their account information by clicking settings, which produces a sidebar on the left of the screen. Here, the user may select if they wish to edit their profile, change their photo, password, or delete their account.



If the user wishes to edit their account, they can update information like name, address, and email

Edit Account Profile

First Name: *	Bruce
Middle Name:	
Last Name: *	Wayne
Address Line 1: *	12345 Wayne Manor
Address Line 2:	
City: *	Gotham
State Name:	New York ▼
Zip Code: *	12345
Email: *	batman@gmail.com
Username:	Batman

To change your photo, the user can either upload a current photo, by accessing the camera on your device, or the user can upload a file. These files can be saved on the set up undertow system.

Profile Photo

Either Take Your Photo or Upload an Existing One

Position yourself and click Capture to take your photo using your computer's camera

 Capture  Redo  Done

Click Choose File to select an existing photo on your computer and then click Upload to upload it

No file chosen

 Upload

 Cancel

For changing a password, the user must first answer security questions: First provide your username, then answer the question, then the user may update their password

Please Enter Your Username

 Submit  Cancel

Please Answer Your Security Question

In what city or town did your mother and father meet?

↑ Submit

⌛ Cancel

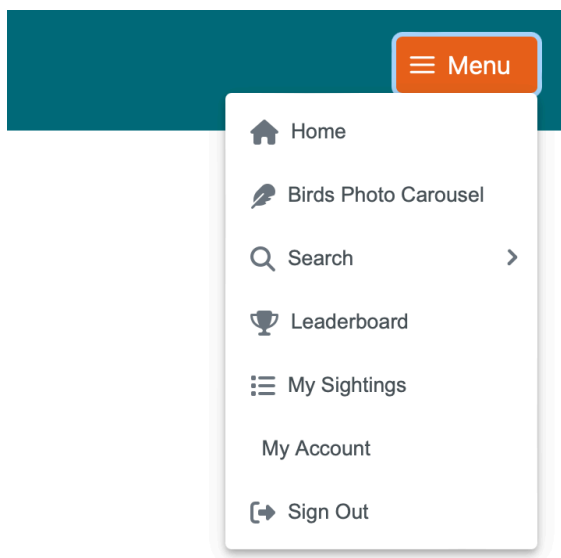
Please Enter Your New Password

Password: ? *

Confirm Password: *

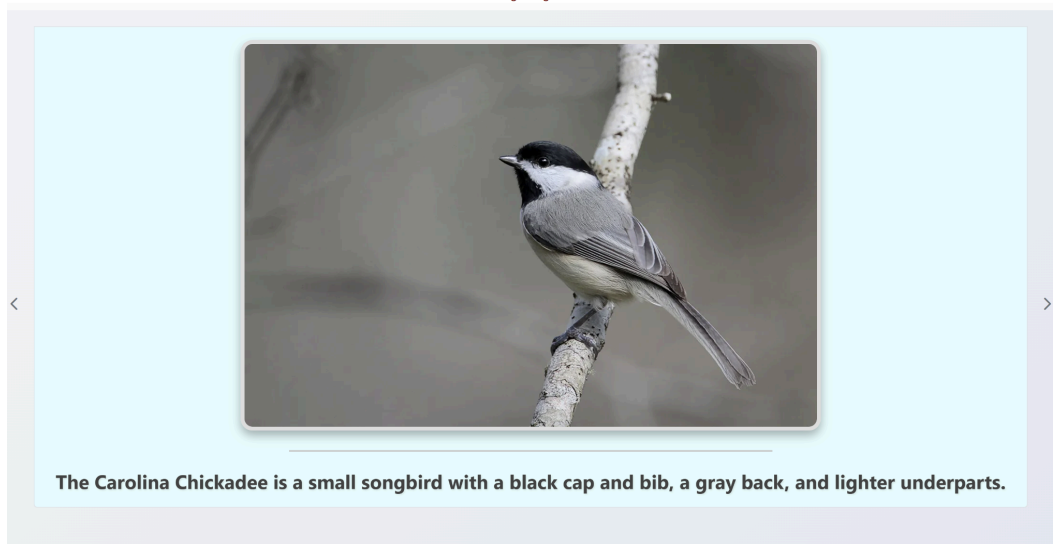
↑ Submit

⌛ Cancel



Upon a successful sign-in, users will now have access to a personal “My Sightings” database to add their bird sightings to their own database.

For the Images Carousel, we have an image slider that cycles through images of birds along with associated facts for more information for the user



The user can compete with other users to view birds. Any birds seen gets added to the user's total birds seen, which is kept track within the user table in the database.

Leaderboard		
<< < 1 > >> 10 ▾		
Rank	Username	Total Birds Seen
1	Batman	24
2	Superman	7
3	BilboBaggins	4
<< < 1 > >> 10 ▾		

In order for a user to find birds to add to their sightings page, a user can do an api search. Upon clicking the menu option for api search, a dialog box will appear that allows the user to enter their desired region. This query returns all recent bird sightings in the area over the past 30 days. Some regions will have large amounts of responses, prompting a slow loading time, while some regions may have no recent sightings.

Search Birds APIs by Region ✕

Region: *

US-CT

↑ Submit

⌛ Cancel

Upon clicking submit, the user is directed to the search results page, which displays a table containing all necessary information that was found on the nearby birds. You can sort or search by different categories.

Bird API Search Results							
<< < 1 2 3 4 5 6 7 8 9 10 > >> 5 ▾							
Common Name ↑↓	Scientific Name ↑↓	Photo	Endangerment Status	Wingspan	Sighting Location	Observation Date	Number of Birds Seen
American Bittern	Botaurus lentiginosus		Low Concern	Unavailable - Unavailable cm	Hammonasset Beach SP	2024-12-08 12:27	1
American Black Duck	Anas rubripes		Low Concern	88 - 95 cm	Nathan Hale Park	2024-12-08 15:39	1
American Coot	Fulica americana		Low Concern	58.4 - 63.5 cm	Beaver Ponds Park	2024-12-08 14:01	5
American Crow	Corvus brachyrhynchos		Low Concern	85 - 100 cm	Riverside Park (Hartford)	2024-12-08 15:39	30
American Goldfinch	Spinus tristis		Low Concern	19 - 22 cm	Suffield WMA	2024-12-08 15:03	1
<< < 1 2 3 4 5 6 7 8 9 10 > >> 5 ▾							

There is a button for displaying the information for just one bird. There is also a button which pops up a dialog for a map of where the bird was found, which allows the user to go to its most recent location. This uses the google maps api, and is displayed using a map feature.

American Goldfinch



Close

If the user wants to add a specific bird, they can click the button for adding a new sighting. This will prefill some information of the desired bird, but also allows the user to add their own information, like when the bird was seen, some notes, and how many were seen (this will update the leaderboard).

Add New Sighting



Bird Common Name:	American Goldfinch
Bird Scientific Name:	Spinus tristis
Bird Endangerment Status:	Low Concern
Bird Sighting Location:	Suffield WMA
Observation Date: *	
Observation Notes: *	
How Many Seen: *	1

Save

Cancel

Within the sightings table, there is an attached foreign key for each sighting. This will associate each sighting with a user. So when a user is logged in, they may perform various queries to search within their own sightings databases. There are 4 types of queries, each time they are clicked, they will have a pop up dialog for input. Type 1 search will contain an accordion panel to select the category.



Database Search

Type	Description	Search
1	Search sighting name, location or how many sightings individually or search for each of it.	Search Type 1
2	Sighting name contains nameQ AND numberOfSightings ≥ numberOfSightingsQ	Search Type 2
3	Sighting location contains locationQ AND numberOfSightings ≥ numberOfSightingsQ	Search Type 3
4	Sighting name contains nameQ AND Sighting location contains locationQ	Search Type 4

Each button has an associated dialog pop up. The user inputs the required information for the query, and is then taken to a new page for the results of that database search.

Type 1 Search

Select Search Category

Enter search string

Submit

Cancel

Type 2 Search

Sighting Name *

Sighting numberOfSightings ≥ *

Submit

Cancel

Type 3 Search

Sighting Location *

Sighting numberOfSightings ≥ *

Submit

Cancel

Type 4 Search

Sighting Name *

Sighting Location *

Submit

Cancel

This is an example query of a search for “pigeon” in the database. The search would return basic information for the birds, and for each bird, you can view additional information with “view”, or you can click “map” to see the exact location it was seen. You can also go back to return to the search query page.

Sighting Search Results					
<< < 1 > >> 10					
Common Name ↑↓	Scientific Name ↑↓	Image	Location ↑↓	Observation Notes	Number of Sightings ↑↓
Rock Pigeon	Columba livia		Piazza San Pietro	Birds	3
<< < 1 > >> 10					
View Map Go Back					

For the user’s personal sightings, their sightings are displayed similar to the api search, listing birds in a table with information like name of the bird, sighting time, location, and how many were seen.

<< < 1 > >> 10 ▾

Name ↑↓ 	Image	Sighting Date and Time ↑↓	Sighting Location ↑↓ 	Observation Notes ↑↓	Number of Birds Seen ↑↓	Wingspan ↑↓	Status ↑↓
American Coot (Fulica americana)		12/4/2024	Falls Mills Lake	A duck on the water	12	58.4cm - 63.5cm	Low Concern
American Goldfinch (Spinus tristis)		12/3/2024	Fort Boykin Historic Park	Bright yellow birds	3	19cm - 22cm	Low Concern
American Oystercatcher (Haematopus palliatus)		12/5/2024	Chincoteague Island	Catching some oysters on the ground	3	Unavailablecm - Unavailablecm	Restricted Range
American White Pelican (Pelecanus erythrorhynchos)		11/25/2024	Hog Island WMA	Some big pelicans catching fish	3	244cm - 290cm	Low Concern
Rock Pigeon (Columba livia)		12/4/2024	Piazza San Pietro	Birds	3	50cm - 67cm	Low Concern

<< < 1 > >> 10 ▾

+ Create 👁 View ✎ Edit 🗑 Delete 📷 Photo 🌐 Map

If the user does not wish to add a bird through the api, they can create a new sighting manually. They can also change a sighting with the edit button. All the buttons listed above show a dialog for each function.

Create New Sighting

Common Name: *

Scientific Name: *

Date and Time: *

Location: *

Latitude: *

Longitude: *

Observation Notes: *

Image URL: *

Number of Birds Seen: *

Minimum Wingspan: *

Edit American White Pelican Data

Common Name: *

Scientific Name: *

Date and Time: *

Location: *

Latitude: *

Longitude: *

Observation Notes: *

Image URL: *

Number of Birds Seen: *

Minimum Wingspan: *

The user can also view a larger version of the bird, allowing them to see more detailed photographs. From the photo’s dialog, the user may print out the photo of the bird. A user can also delete a sighting from the database, with a confirmation dialog for extra security.

American Goldfinch



Close Print

Deletion Confirmation



Are you sure you want to permanently delete the selected sighting American White Pelican from the database?

No

Yes

On the footer of each page, there is a button to open a dialog of our ai chat bot. From here, you can ask questions about birds. If you have a certain bird selected within the sightings database, you can click the preset questions for the diet, fun facts, and predators. When you are done, you can also clear the chat.

Feather Finder ChatBot



Feather Finder: What can I help with?

Type your message...

Send

Selected Bird's Diet

Selected Bird Fun Facts

Selected Bird's Predators



Clear Chat

To execute a “Selected Bird” question, navigate to the “my sightings” tab [which will only be present if a user is signed in], select a bird in your sightings database, and then click one of the 3 options to send a query to Gemini with an associated bird’s information bundled in your request header, otherwise, you will receive a message from the server saying that you must make a bird selection before attempting to utilize that feature.

8. CONCLUSIONS

Overall, Team 15 implemented the Feather Finder web-application by adopting a software engineering paradigm commonly known as the AGILE methodology. To adhere to this paradigm, Team 15 utilized GitHub to handle version control of the project over time, allowing each member to contribute at their own pace. To keep members at a steady pace, Team 15 held weekly meetings to hold virtual "stand ups" where each member could share what they've accomplished, what they're currently working on or stuck on, and what each member plans to finish before the next meeting. Keeping each member updated on the work they completed was a driving factor in the success of the delivery of the web-application. Team

15 had a minimum viable product at each phase of development to ensure the main idea of the Feather Finder application was able to be visualized and interacted with at each step of the way towards the delivery of the software. In terms of application functionality, Team 15 delivered every single feature that was promised in the initial phases of the report draft, while also adding additional features.

9. SUBMISSION INSTRUCTIONS

Please carefully follow the instructions below in submitting your project:

1. Microsoft Word or PDF version of this report file is acceptable.
2. Deploy your cloud software application to your Amazon EC2 virtual private server.
3. Using the deployed software, each team member is required to create an account and generate *meaningful content* in the database for testing and grading. **You will be penalized for not having sufficient content.** In the case of two-factor authentication, turn it off for each user account for grading. List the usernames and passwords in the *User Accounts* section on page ii.
4. Create a new folder named **CS3754 Project Team N** where N = your team number. Do not replace the spaces in the filename with anything else.
5. Place the following in the created folder:
 - a. This file: *CS3754 Team N Semester Project Report.docx* or *.pdf* (Replace N with your team number. Do not change this filename and do not remove the spaces in the filename!)
 - b. Your **IntelliJ IDEA Ultimate project folder** containing your entire documented software application.
 - c. Your original Database SQL file to create the tables in your database and populate it with sample data.
6. Right click the folder and compress it to generate the ZIP file **CS3754 Project Team N.zip**.
7. Verify the ZIP file: Unzip the compressed file on another computer and make sure that the ZIP file is properly created.
8. Upload your ZIP file to your account in **OneDrive**, right click the uploaded file and select **Share** from the pop-up menu, enter **balci@vt.edu**, add a note as CS3754 Project Team N Submission, and click Send before the due date.
9. Only one team member should submit the project on behalf of the team **by copying the team members. Late submissions are not allowed.**
10. If your project app is a multi-user app or an app that requires demonstration to evaluate its functionality, then get an appointment with Dr. Balci before the due date to give a hands-on demo so that your app can be graded properly.

10. PERCENTAGES OF CONTRIBUTION

We hereby certify that the list of contributions and the corresponding percentages of contribution specified below truly reflect the actual contributions of the team members.

(Write your name as your signature)

<i>Student Name & List of Contributions</i>	<i>% Contributed</i>	<i>Signature</i>
Logan Kemprowski's Contributions: 1. API Controller 2. API front end for search dialog 3. API front end for search results 4. CSS styling	16.666%	Logan Kemprowski
Chaebin Kim's Contributions: 1. Sighting Controller 2. Entity/Facade Beans for Sighting 3. Front-end functions for user sighting lists 4. Made Sighting table in SightingsDB	16.666%	Chaebin Kim
Hamza Iqbal's Contributions: 1. Login Page and Database 2. Functionality to track user-specific Bird count 3. Leaderboard Page (Controller, Front-End, Styling) 4. Setting up project files and GitHub	16.666%	Hamza Iqbal
Solaiman Ibrahim's Contributions: 1. Gemini Controller 2. AI ChatBot UI 3. Selected Sighting Bird ChatBot Queries	16.666%	Solaiman Ibrahim
Samarth Jain's Contributions: 1. Facade Bean for Sighting (with search) 2. Database Search UI and Functionality 3. Print to Save as PDF in List and DatabaseSearchResults	16.666%	Samarth Jain
Quang Dao Le's Contributions: 1. Index Page and Galleria 2. Photo Carousel 3. Template Web 4. Create UI Wireframe	16.666%	Quang Dao Le
Sum of Percentages:	100%	

11. CALCULATION OF GRADES

Grade Calculation.xls						
	A	B	C	D	E	F
1		Grade Given for the Project	Student 1 Percentage of Contribution	Student 2 Percentage of Contribution	Student 3 Percentage of Contribution	
2	Example:	86	40%	25%	35%	100%
3						
4	Student 1	91.73	= B2 + B2 * (C2 - 33.333%)			
5	Student 2	78.83	= B2 + B2 * (D2 - 33.333%)			
6	Student 3	87.43	= B2 + B2 * (E2 - 33.333%)			
7						

Additional percentage of contribution cannot exceed 10%.

If a situation arises where a student is doing more than 10% extra work, Dr. Balci must be informed immediately, not at the end of the semester.

12. TEAM MEMBER RULES

1. It is the professional obligation of each team member **not** to drop or withdraw from the course after committing to be a team member.
2. Each team member is expected to contribute equally.
3. You shall submit percentages of contribution together with a list of each student's individual contributions with signatures of all team members. In case of disagreement, you shall submit it separately with your rationale. (Write your name as representing your signature.)
4. Grades shall be determined based on the percentages of contribution.
 - a. If you contribute more than your equal share, then your grade shall be increased based on the extra percentage of contribution, which cannot be more than 10%.
 - b. If you contribute less than your equal share, then your grade shall be decreased accordingly.
 - c. The extra percentage of contribution shall not be more than 10%. Dr. Balci shall be notified immediately if a situation arises where a student needs to contribute more than 10% extra.
5. A team member who does not cooperate with other team members for conducting the project with equal contribution shall be penalized. **Doing more work than agreed upon by the team and claiming extra contribution shall not be acceptable.** Cooperation is essential!

13. GRADING SHEET

CS3754 Semester Project Grading: Team N				
		Points	Earned	Notes
	Requirements Specification	5		Minimum 15 Functional Requirements (3 per team member for a team of 5) Minimum 10 Non-Functional Requirements (2 per team member for a team of 5)
	Architecture Specification	5		Revised diagram in given Cloud Software Architecture.pptx with description of each API
	Software Design Specification	20		(10 pts): Minimum 15 annotated easy-to-read balsamiq.com UI wireframes (3 per team member) (6 pts): MySQL database easy-to-read ER diagram created with dbdesigner.net (4 pts): JSON file data structure for each API with used parts marked
	Penalty for Missing Required Features			(-5) App name contains Team or meaningless (-15) No MySQL database used for app data storage and retrieval (-15) No MySQL database complex searches implemented properly (-15) No API is used with search query entered by the user (-75) Not running on Amazon EC2 VPS. The submission is rejected. 25 pts given for the effort. (-10) Empty list (e.g., favorites) with no example content of at least 10 items (-15) No User Account creation and maintenance (-10) No template-based UI with header and footer (-10) Number of project team meeting minutes < 5 and/or their content lacks sufficient detail (-X) Other missing features typically implemented in tutorials
	Software Features			
	Quantity of Features	20		Number of features properly implemented from the list of App Features Learned in Course Tutorials (.../StudentsOnlyProject/LearnedFeatures.html) + new features if implemented
	Variety of Features	20		Variety of features properly implemented from the list of App Features Learned in Course Tutorials (.../StudentsOnlyProject/LearnedFeatures.html) + new features if implemented
	Complexity of Features	20		Difficulty of creating each feature. Does it take minimum 120 hours (for 5 students) to develop the app? Complexity is judged w.r.t. minimum 120 hours time requirement.
	Software Documentation	5		Meaningful and informative documentation, similar to the course tutorials, showing that you understand what the code is doing.
	Project Report Quality	5		(-5) Instructions in red are not deleted in the final version of the report. Title page, table of contents, styles, formatting, hyperlinks, etc.
	Total Points	100	0.00	
		% Cont.	Grade	
	Team Members:			Minimum time required: 6 hrs/week x 4 weeks x 5 students = 120 hrs
✓	1. Student Full name 1	20.000%	0.00	
✓	2. Student Full name 2	20.000%	0.00	
✓	3. Student Full name 3	20.000%	0.00	
✓	4. Student Full name 4	20.000%	0.00	
✓	5. Student Full name 5	20.000%	0.00	
		100.000%		

REFERENCES

- Balci, O. (2024), “CS3754 Cloud Software Development Course Website,” <https://manta.cs.vt.edu/cs3754>
- eBird API 2.0 (2021), “eBird API 2.0,” <https://documenter.getpostman.com/view/664302/S1ENWwy59#3d2a17c1-2129-475c-b4c8-7d362d6000cd>
- Gemini API (2024), “Gemini API | Google AI for Developers,” <https://ai.google.dev/gemini-api/docs>
- Eclipse Foundation (2024), “Jakarta EE,” <https://jakarta.ee/>
- JetBrains (2024), “Intelligent Java Integrated Development Environment Advanced (IntelliJ IDEA),” <https://www.jetbrains.com/idea/>
- nuthatch (2024), “nuthatch API,” <https://nuthatch.lastelm.software/swagger.html>
- MySQL (2024), “MySQL Open Source Relational Database Management System,” <https://www.mysql.com/>
- PrimeFaces (2024), “PrimeFaces UI Component Library for JSF,” <https://www.primefaces.org/>
- WildFly (2024), “WildFly Jakarta EE Application Server,” <https://www.wildfly.org/>

APPENDIX A: MEETING MINUTES

Meeting Number 1

<i>Date & Duration:</i>	10/27/24 – 1.5 hrs
<i>Location:</i>	Virtual
<i>Members Present:</i>	Logan, Samarth, Hamza, Dao, Chaebin
<i>Members Absent:</i>	Solaiman
<i>Discussions:</i>	Deciding on the problem to solve for the web application, what features to implement, and the proper API to use to solve the proposed problem. We each proposed a different idea, and discussed which one would work best, and how we could incorporate the different features into each. After discussing which idea worked best, we went further in depth as to how to use the API's and any potential new features.
<i>Decisions Made:</i>	We decided to go with Logan's idea of a bird watching app, which allows users to share birds they saw, and compete with other users on who saw the most birds. We decided on the API's that we would be using, such as eBird, nuthatch, and the openAI APIs. We then filled out the problem proposal sections for submission.
<i>Work Assignments:</i>	Everyone was assigned to do more research on the proposed API's and features before the next meeting, to see how best to implement them. Everyone should also come up with an idea for a name for the web application before the next meeting.
<i>Minutes Prepared By:</i>	Logan Kemprowski

Meeting Number 2

<i>Date & Duration:</i>	11/10/24 - 30 min
<i>Location:</i>	Virtual
<i>Members Present:</i>	Logan, Chaebin, Hamza, Dao, Samarth, Solaiman
<i>Members Absent:</i>	None
<i>Discussions:</i>	Discussed how to plan out the presentation. We decided what to put on each slide, and which parts each person would be working on / presenting.
<i>Decisions Made:</i>	We gave roles for each person in the presentation: Hamza: Problem Specification Logan: APIs and database slides Samarth: Learned and new features planned Dao: Functional Requirements Chaebin: Nonfunctional Requirements Solaiman: Architecture We decided for a check in meeting on Tuesday (11/12/24)
<i>Work Assignments:</i>	Work on each person's individual slides until Tuesday, complete slides before Thursday morning.
<i>Minutes Prepared By:</i>	Logan Kemprowski

Meeting Number 3

<i>Date & Duration:</i>	11/12/24 - 20 min
<i>Location:</i>	Virtual
<i>Members Present:</i>	Logan, Chaebin, Hamza, Solaiman
<i>Members Absent:</i>	Samarth, Dao
<i>Discussions:</i>	Met to have a progress check on the presentation, and did some practice runs in order to see the order of who is presenting, and how long the presentation would take to ensure that we stay within the time limit.
<i>Decisions Made:</i>	We reviewed the currently finished slides, and made needed adjustments. We couldn't do a run through as there were unfinished slides and absent members, so we will time ourselves individually to ensure we stay within the 5 minute time limit. We planned another meeting for 11/13/24 for a final practice before submission.
<i>Work Assignments:</i>	Everyone will time themselves individually and finish their assigned slides.
<i>Minutes Prepared By:</i>	Logan Kemprowski

Meeting Number 4

<i>Date & Duration:</i>	11/13/24 - 30 min
<i>Location:</i>	Virtual
<i>Members Present:</i>	Logan, Chaebin, Hamza, Samarth, Dao
<i>Members Absent:</i>	Solaiman
<i>Discussions:</i>	Did practice runs of the presentation and made last minute changes in order to fit the proper time. Discussed order of who was speaking, and who would take over the last slide for Solaiman as he will be absent for presentation day.
<i>Decisions Made:</i>	Finalized the order of speaking. Decided Chaebin would take over the last slide. Got the time for the presentation down to 4 min, 30 sec.
<i>Work Assignments:</i>	Practice individual parts for presentation day
<i>Minutes Prepared By:</i>	Logan Kemprowski

Meeting Number 5

<i>Date & Duration:</i>	11/17/24 - 20 min
<i>Location:</i>	Virtual
<i>Members Present:</i>	Logan, Chaebin, Hamza, Samarth, Dao, Solaiman
<i>Members Absent:</i>	None
<i>Discussions:</i>	Discussed the second presentation and distributed roles and decided when the next meeting would be.
<i>Decisions Made:</i>	Samarth and Dao will do the UI wireframing slides. Hamza will take Local Birds Database. Chaebin will do Birds Seen Database. Logan and Solaiman will do the JSON data for the APIs. Logan will take EBird and nuthatch, Solaiman will work on Gemini and GoogleMaps. We will have a progress check meeting on Tuesday.

<i>Work Assignments:</i>	Work on assigned slides until Tuesday before the meeting so we can have time to check in and do peer review.
<i>Minutes Prepared By:</i>	Logan Kemprowski

Meeting Number 6

<i>Date & Duration:</i>	11/19/24 - 20 min
<i>Location:</i>	Virtual
<i>Members Present:</i>	Logan, Chaebin, Hamza, Samarth, Dao
<i>Members Absent:</i>	Solaiman
<i>Discussions:</i>	Went over current progress, and discussed any clarification on the API call information, as well as database design. We also covered the progress done on the wireframe so far, with Samarth streaming his current design.
<i>Decisions Made:</i>	We finalized what API data will be called from the birds APIs. We also covered how the two databases will be connected, and when our next meeting will be (11/20/24).
<i>Work Assignments:</i>	Each member will finish their assigned parts, and practice presenting each section before tomorrow before our final test run and submission.
<i>Minutes Prepared By:</i>	Logan Kemprowski

Meeting Number 7

<i>Date & Duration:</i>	11/20/24 - 35 min
<i>Location:</i>	Virtual
<i>Members Present:</i>	Logan, Chaebin, Hamza, Samarth, Dao, Solaiman
<i>Members Absent:</i>	None
<i>Discussions:</i>	Discussed any final changes to be made on the presentation. Did multiple practice runs, and decided which parts everyone would be presenting.
<i>Decisions Made:</i>	We made some changes to the database slides, and cut out unnecessary parts to the JSON API slides. We decided who would be presenting each slide, and got a final time of ~4:15 minutes.
<i>Work Assignments:</i>	Everyone will individually practice their own slides before the presentation.
<i>Minutes Prepared By:</i>	Logan Kemprowski

Meeting Number 8

<i>Date & Duration:</i>	12/2/24 - 25 min
<i>Location:</i>	Virtual
<i>Members Present:</i>	Logan, Chaebin, Hamza, Samarth, Dao, Solaiman
<i>Members Absent:</i>	None
<i>Discussions:</i>	Discussed any progress being done to the project. Talked through everyone's roles, and made changes depending on how much has been done, and who doesn't have enough work.

<i>Decisions Made:</i>	Figured out who would be working on parts like the ranking system, and clarified the databases we would need, removing a favorites database. Also decided on the position of the ai chatbot feature.
<i>Work Assignments:</i>	Everyone will individually work on their own parts until we meet again for another progress check.
<i>Minutes Prepared By:</i>	Logan Kemprowski

Meeting Number 9

<i>Date & Duration:</i>	12/4/24 - 1 hour
<i>Location:</i>	Virtual
<i>Members Present:</i>	Logan, Chaebin, Hamza, Samarth, Dao, Solaiman
<i>Members Absent:</i>	None
<i>Discussions:</i>	Discussed progress being made on the project and assigned who will do the leaderboard. Decided the order of presentation.
<i>Decisions Made:</i>	Order of presentation and next meeting time.
<i>Work Assignments:</i>	Prepare for presentation
<i>Minutes Prepared By:</i>	Chaebin Kim

Meeting Number 10

<i>Date & Duration:</i>	12/5/24 - 30 min
<i>Location:</i>	Virtual
<i>Members Present:</i>	Logan, Chaebin, Hamza, Samarth, Solaiman
<i>Members Absent:</i>	Dao
<i>Discussions:</i>	Did multiple practice runs for the demo. Decided which parts everyone would be presenting by practicing with the actual app.
<i>Decisions Made:</i>	Adding names to the application and some edits in the database search and database list.
<i>Work Assignments:</i>	Practice own part of the presentation
<i>Minutes Prepared By:</i>	Chaebin Kim

Meeting Number 11

<i>Date & Duration:</i>	12/8/24 - 30 min
<i>Location:</i>	Virtual
<i>Members Present:</i>	Logan, Chaebin, Hamza, Samarth, Solaiman, Dao
<i>Members Absent:</i>	None
<i>Discussions:</i>	Divided up parts for the final report and went over what else needed to be done before final deployment of the web application.
<i>Decisions Made:</i>	Assigned individual parts to each person to work on before the deadline.
<i>Work Assignments:</i>	Complete the report, each person will take their own individual parts.
<i>Minutes Prepared By:</i>	Logan Kemprowski

Meeting Number 12

<i>Date & Duration:</i>	12/11/24 -
<i>Location:</i>	Virtual
<i>Members Present:</i>	Logan, Chaebin, Hamza, Samarth, Solaiman, Dao
<i>Members Absent:</i>	None
<i>Discussions:</i>	Went through the report together and edited it if anything needed to be added.
<i>Decisions Made:</i>	Deployed the application and submitted the report
<i>Work Assignments:</i>	Finished Project, no deployment
<i>Minutes Prepared By:</i>	Chaebin Kim